Massey University
COLLEGE OF SCIENCES

Relational Set Operators

Relational algebra

Defines theoretical way of manipulating table contents using relational operators.

Use of relational algebra operators - unary and binary - on existing relations produces new relations

Unary – applied on one relation

SELECTION and PROJECTION

Binary – applied on two relations

PRODUCT, JOINS, UNION, MINUS, INTERSECT

Massey University
COLLEGE OF SCIENCES

SELECTION

The relational operator SELECT (also sometimes referred as RESTRICT) can be used to list all of the row values or it can be used to return only those values that match a specified condition)

The SELECT operator works on a single relation (table) and defines an operation that contains only those tuples (rows) that satisfy a specified condition.

Table – PRODUCT (P_Code, P_Description, Price)

e.g. selecting all the products

e.g. selecting all the products and their descriptions

Massey University
COLLEGE OF SCIENCES

SELECT Operator

P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

Table – PRODUCT (P_Code, P_Description, Price)

SELECT * FROM PRODUCT

Now say we want to restrict the result to view a list of products with Price < $2.00

SELECT P_CODE, P_DESCRIPTION, PRICE FROM PRODUCT
WHERE PRICE < 2
(Other operators are >, >=, <=, PRICE BETWEEN 1.5 AND 2.0, PRICE IN (1.5, 2.0))
(Or P_DESCRIPTION LIKE 'CABLE%' or P_DESCRIPTION NOT LIKE '_AB%')

Massey University
COLLEGE OF SCIENCES

SELECT Operator

FIGURE 3.5 **SELECT**

Original table

P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

SELECT ALL yields →

New table

P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

SELECT only PRICE less than $2.00 yields →

P_CODE	P_DESCRIPT	PRICE
213345	9v battery	1.92
254467	100W bulb	1.47

SELECT only P_CODE = 311452 yields →

P_CODE	P_DESCRIPT	PRICE
311452	Powerdrill	34.99

Massey University
COLLEGE OF SCIENCES

PROJECTION

P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

Projection returns all values for selected attributes excluding duplicates. It returns a vertical subset of a relation excluding duplicates

Again this uses a single table and we use the SELECT operator to return the values

Table – PRODUCT (P_Code, P_Description, Price)

SELECT DISTINCT PRICE FROM PRODUCT

PRICE
5.26
25.15
10.99
1.92
1.47
34.99

Massey University
COLLEGE OF SCIENCES

Select and Project

Select (or Restrict) and Project are UNARY operators.

They are performed on individual tables and not on pairs of tables.

Note that even if two tables are joined (thru join operations), then the result set is a single table and the Select or Project operator is performed on that single table.

Massey University
COLLEGE OF SCIENCES

CARTESIAN PRODUCT

The Cartesian product indicates a many-to-many relationship, i.e. it indicates that the WHERE clause has no joining columns or that multiple rows from one table match multiple rows in another table.

PRODUCT (P_Code, P_Description, Price)

LOCATION (Store, Aisle, Shelf)

```
SELECT COUNT (*) FROM PRODUCT, LOCATION
// if PRODUCT has 6 items and LOCATION has 3 items, then the result is 18
SELECT P.P_CODE, P.P_DESCRIPTION, P_PRICE,
        L.STORE STORE, L.AISLE, L.SHELF
FROM PRODUCT P, LOCATION L
```

Massey University
COLLEGE OF SCIENCES

CARTESIAN PRODUCT Operator

P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

PRODUCT

STORE	AISLE	SHELF
23	W	5
24	K	9
25	Z	6

yields

P_CODE	P_DESCRIPT	PRICE	STORE	AISLE	SHELF
123456	Flashlight	5.26	23	W	5
123456	Flashlight	5.26	24	K	9
123456	Flashlight	5.26	25	Z	6
123457	Lamp	25.15	23	W	5
123457	Lamp	25.15	24	K	9
123457	Lamp	25.15	25	Z	6
123458	Box Fan	10.99	23	W	5
123458	Box Fan	10.99	24	K	9
123458	Box Fan	10.99	25	Z	6
213345	9v battery	1.92	23	W	5
213345	9v battery	1.92	24	K	9
213345	9v battery	1.92	25	Z	6
311452	Powerdrill	34.99	23	W	5
311452	Powerdrill	34.99	24	K	9
311452	Powerdrill	34.99	25	Z	6
254467	100W bulb	1.47	23	W	5
254467	100W bulb	1.47	24	K	9
254467	100W bulb	1.47	25	Z	6

QUESTION

As you can see that the Cartesian product have combined tuples which have no association with each other.

So, what is the purpose of the Cartesian Product?

Massey University
COLLEGE OF SCIENCES

Union Compatible

In order for two relations to be union-compatible, both must have the same number of attributes (columns) and corresponding attributes (columns) must have the same domain.

- If first relation has 3 attributes then second relation must also have 3 attributes.
- An attribute's domain is the set of allowable values for that attribute. Thus all attributes of both relations must have same domain

  i.e., The first attribute of the first relation must have the same domain as the first attribute of the second relation, the second attribute of the first relation must have the same domain as the second attribute of the second relation, and so on.

Massey University
COLLEGE OF SCIENCES

UNION and UNION ALL

When we want to combine a list of two tables into one single result set.

However, the two tables should be union –compatible (i.e. column and domain must be identical)

Two business divisions create electrical appliances. i.e. DIY appliances and Domestic appliances

Table – DIY_PRODUCT (P_Code, P_Description, Price)

Table – DOMESTIC_PRODUCT (P_Code, P_Description, Price)

```
SELECT P_CODE, P_DESCRIPTION, PRICE FROM DIY_PRODUCT
UNION
SELECT P_CODE, P_DESCRIPTION, PRICE FROM DOMESTIC_PRODUCT
```

Using UNION ALL will also include duplicate products (if any)

Massey University
COLLEGE OF SCIENCES

UNION Operator

P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

UNION

P_CODE	P_DESCRIPT	PRICE
345678	Microwave	160.00
345679	Dishwasher	500.00

yields

P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99
345678	Microwave	160
345679	Dishwasher	500

UNION operator combines two or more sets of data to produce a single set of data.

Massey University
COLLEGE OF SCIENCES

Changing Column Names

UNION operator will also work if number of columns and datatype of the columns match (even though the column names may not match). In such cases, it is good to use an change the column name – to something which is common for both columns,

SELECT <ATTRIBUTE_NAME> <ALIAS_NAME>,

TEACHER (TEACHER_ID, TEACHER_NAME, TEACHER_DOB)
STUDENT (STUDENT_ID, STUDENT_NAME, STUDENT_DOB)

```
SELECT TEACHER_ID PERSON_ ID, TEACHER_NAME PERSON_NAME
FROM TEACHER
UNION
SELECT STUDENT_ID, STUDENT_NAME FROM STUDENT
```

Massey University
COLLEGE OF SCIENCES

MINUS and INTERSECT Operator

The MINUS set operator subtracts one set of data from another identifying what data exists in one table but not the other.

The INTERSECT operator is the intersection of sets of data, identifying data common to both.

Table – DIY_PRODUCT (P_Code, P_Description, Price)
Table – DOMESTIC_PRODUCT (P_Code, P_Description, Price)

```
SELECT P_CODE FROM DIY_PRODUCT
MINUS
SELECT P_CODE FROM DOMESTIC_PRODUCT
```

```
SELECT P_CODE FROM DIY_PRODUCT
INTERSECT
SELECT P_CODE FROM DOMESTIC_PRODUCT
```

Massey University
COLLEGE OF SCIENCES

INTERSECT and MINUS/ DIFFERENCE

FIGURE 3.7 INTERSECT

STU_FNAME	STU_LNAME
George	Jones
Jane	Smith
Peter	Robinson
Franklin	Johnson
Martin	Lopez

INTERSECT

EMP_FNAME	EMP_LNAME
Franklin	Lopez
William	Turner
Franklin	Johnson
Susan	Rogers

yields

STU_FNAME	STU_LNAME
Franklin	Johnson

SOURCE: Course Technology/Cengage Learning

FIGURE 3.8 DIFFERENCE

STU_FNAME	STU_LNAME
George	Jones
Jane	Smith
Peter	Robinson
Franklin	Johnson
Martin	Lopez

DIFFERENCE

EMP_FNAME	EMP_LNAME
Franklin	Lopez
William	Turner
Franklin	Johnson
Susan	Rogers

yields

STU_FNAME	STU_LNAME
George	Jones
Jane	Smith
Peter	Robinson
Martin	Lopez

SOURCE: Course Technology/Cengage Learning

Note: A – B is not equal to B – A

INTERSECT – determines common values between two sets.

MINUS (DIFFERENCE) – subtracts one set of data from another, identifying what data exists in one table but not in the other

Massey University
COLLEGE OF SCIENCES

Question Time

Table name: BOOTH

BOOTH_PRODUCT	BOOTH_PRICE
Chips	1.5
Cola	1.25
Energy Drink	2

Table name: MACHINE

MACHINE_PRODUCT	MACHINE_PRICE
Chips	1.25
Chocolate Bar	1
Energy Drink	2

What is the RESULT....

MACHINE UNION BOOTH ---??

MACHINE UNION ALL BOOTH ---??

MACHINE INTERSECT BOOTH ---??

BOOTH INTERSECT MACHINE ---??

MACHINE DIFFERENCE BOOTH --??

BOOTH DIFFERENCE MACHINE ---??

Massey University
COLLEGE OF SCIENCES

SQL Scripts to create tables

```
/* -- */
CREATE TABLE BOOTH (
BOOTH_PRODUCT  VARCHAR2(50),
BOOTH_PRICE    NUMBER(5,2)
);

INSERT INTO BOOTH VALUES ('Chips', 1.5);
INSERT INTO BOOTH VALUES ('Cola', 1.25);
INSERT INTO BOOTH VALUES ('Energy Drink', 2);

/* -- */
CREATE TABLE MACHINE (
MACHINE_PRODUCT  VARCHAR2(50),
MACHINE_PRICE    NUMBER(5,2)
);

INSERT INTO MACHINE VALUES ('Chips', 1.25);
INSERT INTO MACHINE VALUES ('Chocolate Bar', 1);
INSERT INTO MACHINE VALUES ('Energy Drink', 2);
```

Table name: BOOTH

BOOTH_PRODUCT	BOOTH_PRICE
Chips	1.5
Cola	1.25
Energy Drink	2

Table name: MACHINE

MACHINE_PRODUCT	MACHINE_PRICE
Chips	1.25
Chocolate Bar	1
Energy Drink	2

Massey University
COLLEGE OF SCIENCES

Syntax Alternatives

IN and NOT IN can be used in place of INTERSECT and MINUS

Table – DIY_PRODUCT (P_Code, P_Description, Price)

Table – DOMESTIC_PRODUCT (P_Code, P_Description, Price)

```
SELECT P_CODE FROM DIY_PRODUCT
MINUS
SELECT P_CODE FROM DOMESTIC_PRODUCT

SELECT P_CODE FROM DIY_PRODUCT
INTERSECT
SELECT P_CODE FROM DOMESTIC_PRODUCT
```

Example:

```
SELECT P_CODE FROM DIY_PRODUCT
WHERE P_CODE NOT IN ( SELECT DISTINCT P_CODE
                      FROM  DOMESTIC_PRODUCT);
```

Massey University
COLLEGE OF SCIENCES

But first – Using Table Aliases

We often use Table Alias to identify the source table from which data are taken

Alias are especially useful when a table must be joined to itself via a recursive query. Aliases help to differentiate the table from itself

Table alias names are arbitrary. It is best to keep name short and simple. Remember, you cannot use reserved words.

Add alias after table name in FROM clause

```
SELECT alias.columnname
FROM tablename alias
```

Massey University
COLLEGE OF SCIENCES

Using TABLE ALIASES for 2 tables

Example –

CUSTOMER (CUS_CODE, CUS_LNAME, CUS_ZIP, AGENT_CODE)
AGENT (AGENT_CODE, AGENT_PHONE)

```
SELECT C.CUS_CODE, A.AGENT_CODE, C.CUS_LNAME
    FROM CUSTOMER C, AGENT A
    WHERE C.AGENT_CODE = A.AGENT_CODE
```

CUSTOMER has alias C, and AGENT has alias A

Table name: CUSTOMER

CUS_CODE	CUS_LNAME	CUS_ZIP	AGENT_CODE
1132445	Walker	32145	231
1217782	Adares	32145	125
1312243	Rakowski	34129	167
1321242	Rodriguez	37134	125
1542311	Smithson	37134	421
1657399	Vanloo	32145	231

Table name: AGENT

AGENT_CODE	AGENT_PHONE
125	6152439887
167	6153426778
231	6152431124
333	9041234445

Massey University
COLLEGE OF SCIENCES

JOINS

Join operation merges tuples (rows) from two relations (tables).

It is a binary operation and allows users to combine two relations in a specified way.

There are different type of JOINS:

1. Theta Join
2. Equijoin or Inner Join
3. Non equijoin
4. Outer Join (left, right)
5. Self-Join

Massey University
COLLEGE OF SCIENCES

Set Operators

FIGURE 3.10 Two tables that will be used in join illustrations

Table name: CUSTOMER

CUS_CODE	CUS_LNAME	CUS_ZIP	AGENT_CODE
1132445	Walker	32145	231
1217782	Adares	32145	125
1312243	Rakowski	34129	167
1321242	Rodriguez	37134	125
1542311	Smithson	37134	421
1657399	Vanloo	32145	231

Table name: AGENT

AGENT_CODE	AGENT_PHONE
125	6152439887
167	6153426778
231	6152431124
333	9041234445

SOURCE: Course Technology/Cengage Learning

Set Operators – We will join meaningful information from two tables into one meaningful result.

CUSTOMER (CUS_CODE, CUS_LNAME, CUS_ZIP, AGENT_CODE)

AGENT (AGENT_CODE, AGENT_PHONE)

Massey University
COLLEGE OF SCIENCES

SQL Scripts for creating tables

AGENT and CUSTOMER

```
CREATE TABLE AGENT (
AGENT_CODE VARCHAR2(8) PRIMARY KEY,
AGENT_PHONE VARCHAR2(50));

CREATE TABLE CUSTOMER (
CUS_CODE VARCHAR2(50),
CUS_LNAME VARCHAR2(50),
CUS_ZIP VARCHAR2(10),
AGENT_CODE VARCHAR2(8) REFERENCES AGENT(AGENT_CODE));

/* insert values */
INSERT INTO AGENT VALUES('125','6152439887');
INSERT INTO AGENT VALUES('167','6153426778');
INSERT INTO AGENT VALUES('231','6152431124');
INSERT INTO AGENT VALUES('333','9041234445');

INSERT INTO CUSTOMER VALUES('1132445','Walker','32145','231');
INSERT INTO CUSTOMER VALUES('1217782','Adares','32145','125');
INSERT INTO CUSTOMER VALUES('1312243','Rakowski','34129','167');
INSERT INTO CUSTOMER VALUES('1321242','Rodriguez','37134','125');
INSERT INTO CUSTOMER VALUES('1542311','Smithson','37134','421');
INSERT INTO CUSTOMER VALUES('1657399','Vanloo','32145','231');
/* -- */
```

Table name: AGENT

AGENT_CODE	AGENT_PHONE
125	6152439887
167	6153426778
231	6152431124
333	9041234445

Table name: CUSTOMER

CUS_CODE	CUS_LNAME	CUS_ZIP	AGENT_CODE
1132445	Walker	32145	231
1217782	Adares	32145	125
1312243	Rakowski	34129	167
1321242	Rodriguez	37134	125
1542311	Smithson	37134	421
1657399	Vanloo	32145	231

Massey University
COLLEGE OF SCIENCES

Theta Join, Equi Join and Natural Join

The Theta Join signifies a join between two relations where the matching criteria could be based upon any of the following comparison operators:

= , < , >, < =, > =, <>

The Equijoin is one special type of Theta Join where the equality (=) condition is used to join the tables concerned.

A Natural Join is the most common join, in which the two relations must have one common attribute (referred to as Join column). e.g. AGENT_CODE is the common column between relations CUSTOMER and AGENT

FIGURE 3.12 Natural join, Step 1: PRODUCT

CUS_CODE	CUS_LNAME	CUS_ZIP	CUSTOMER.AGENT_CODE	AGENT.AGENT_CODE	AGENT_PHONE
1132445	Walker	32145	231	125	6152439887
1132445	Walker	32145	231	167	6153426778
1132445	Walker	32145	231	231	6152431124
1132445	Walker	32145	231	333	9041234445
1217782	Adares	32145	125	125	6152439887
1217782	Adares	32145	125	167	6153426778
1217782	Adares	32145	125	231	6152431124
1217782	Adares	32145	125	333	9041234445
1312243	Rakowski	34129	167	125	6152439887
1312243	Rakowski	34129	167	167	6153426778
1312243	Rakowski	34129	167	231	6152431124
1312243	Rakowski	34129	167	333	9041234445
1321242	Rodriguez	37134	125	125	6152439887
1321242	Rodriguez	37134	125	167	6153426778
1321242	Rodriguez	37134	125	231	6152431124
1321242	Rodriguez	37134	125	333	9041234445
1542311	Smithson	37134	421	125	6152439887
1542311	Smithson	37134	421	167	6153426778
1542311	Smithson	37134	421	231	6152431124
1542311	Smithson	37134	421	333	9041234445
1657399	Vanloo	32145	231	125	6152439887
1657399	Vanloo	32145	231	167	6153426778
1657399	Vanloo	32145	231	231	6152431124
1657399	Vanloo	32145	231	333	9041234445

FIGURE 3.13 Natural join, Step 2: SELECT

CUS_CODE	CUS_LNAME	CUS_ZIP	CUSTOMER.AGENT_CODE	AGENT.AGENT_CODE	AGENT_PHONE
1217782	Adares	32145	125	125	6152439887
1321242	Rodriguez	37134	125	125	6152439887
1312243	Rakowski	34129	167	167	6153426778
1132445	Walker	32145	231	231	6152431124
1657399	Vanloo	32145	231	231	6152431124

FIGURE 3.14 Natural join, Step 3: PROJECT

CUS_CODE	CUS_LNAME	CUS_ZIP	AGENT_CODE	AGENT_PHONE
1217782	Adares	32145	125	6152439887
1321242	Rodriguez	37134	125	6152439887
1312243	Rakowski	34129	167	6153426778
1132445	Walker	32145	231	6152431124
1657399	Vanloo	32145	231	6152431124

Massey University
COLLEGE OF SCIENCES

INTERSECT and EQUIJOIN

Table – DIY_PRODUCT (P_Code, P_Description, Price)

Table – DOMESTIC_PRODUCT (P_Code, P_Description, Price)

INTERSECT

```
SELECT P_CODE, P_DESCRIPTION FROM DIY_PRODUCT
INTERSECT
SELECT P_CODE, P_DESCRIPTION FROM DOMESTIC_PRODUCT
```

EQUIJOIN

```
SELECT P1. P_CODE, P1. P_DESCRIPTION
FROM DIY_PRODUCT P1, DOMESTIC_PRODUCT P2
WHERE P1.P_CODE = P2.P_CODE
```

Both will return same result

Massey University
COLLEGE OF SCIENCES

INNER JOIN

Inner Join is used to express a join that satisfies the join condition.
It returns rows corresponding to a matched condition.

The keyword INNER is optional and typically omitted.

```
SELECT C.CUS_CODE, AGENT_CODE, C.CUS_LNAME
FROM CUSTOMER C JOIN AGENT A
USING (AGENT_CODE);
```

If column names are different, then we use ON condition

```
SELECT C.CUS_CODE, A.AGENT_CODE, C.CUS_LNAME
FROM CUSTOMER C JOIN AGENT A
ON (C.AGENT_CODE = A.AGENT_CODE)
WHERE CUS_LNAME LIKE 'Va%';
```

Though here AGENT_CODE is same in both tables

Massey University
COLLEGE OF SCIENCES

OUTER JOINS

An Outer Join returns results of Inner Join (i.e. matched rows) and also includes unmatched rows from both tables

Left Outer Join - Rows in the table to the left side of the JOIN are to be listed along with matching columns of the other table (which is on the right side)

Right Outer Join - This is essentially same as the Left Outer Join, just the rows in the table to the right side of the JOIN are to be listed along with matching columns of the other table (which is on the left side)

CUSTOMER (CUS_CODE, CUS_LNAME, CUS_ZIP, AGENT_CODE)
AGENT (AGENT_CODE, AGENT_PHONE)

Massey University
COLLEGE OF SCIENCES

LEFT OUTER JOIN Operator

FIGURE 3.14 Left outer join

CUS_CODE	CUS_LNAME	CUS_ZIP	CUSTOMER.AGENT_CODE	AGENT.AGENT_CODE	AGENT_PHONE
1217782	Adares	32145	125	125	6152439887
1321242	Rodriguez	37134	125	125	6152439887
1312243	Rakowski	34129	167	167	6153426778
1132445	Walker	32145	231	231	6152431124
1657399	Vanloo	32145	231	231	6152431124
1542311	Smithson	37134	421		

SOURCE: Course Technology/Cengage Learning

All rows of the table on the left of JOIN are listed – even those with no matching AGENT_CODE

```
SELECT C.CUS_CODE, C.CUS_LNAME, C.CUS_ZIP,
       A.AGENT_CODE, A.AGENT_PHONE
FROM CUSTOMER C LEFT OUTER JOIN AGENT A
  ON C.AGENT_CODE  = A.AGENT_CODE;
```

Massey University
COLLEGE OF SCIENCES

RIGHT OUTER JOIN Operator

FIGURE 3.15 Right outer join

CUS_CODE	CUS_LNAME	CUS_ZIP	CUSTOMER.AGENT_CODE	AGENT.AGENT_CODE	AGENT_PHONE
1217782	Adares	32145	125	125	6152439887
1321242	Rodriguez	37134	125	125	6152439887
1312243	Rakowski	34129	167	167	6153426778
1132445	Walker	32145	231	231	6152431124
1657399	Vanloo	32145	231	231	6152431124
				333	9041234445

SOURCE: Course Technology/Cengage Learning

All rows of the table on the right of JOIN are listed – even those with no matching CUSTOMER_CODE

```
SELECT C.CUS_CODE, C.CUS_LNAME, C.CUS_ZIP,
       A.AGENT_CODE, A.AGENT_PHONE
FROM CUSTOMER C RIGHT OUTER JOIN AGENT A
   ON C.AGENT_CODE  = A.AGENT_CODE
```

Massey University
COLLEGE OF SCIENCES

SELF-JOIN

A Self-Join joins a table to itself by pretending there are different tables involved. This is accomplished by using table aliases, i.e. one table has one alias, and the same table also has another alias.

Useful for checking inconsistencies in data or on recursive relations

COURSE (COURSE_NO, DESCRIPTION, PREREQUISITE)

```
SELECT C1.COURSE_NO, C1.DESCRIPTION COURSE_DESCRIPTION,
    C1.PREREQUISITE, C2.DESCRIPTION PREREQ_DESCRIPTION
FROM COURSE C1 JOIN COURSE C2
ON (C1.PREREQUISITE = C2. COURSE_NO);
```

Note - You can also format columns COURSE_DESCRIPTION and PREQ_DESCRIPTION

Massey University
COLLEGE OF SCIENCES

NON EQUIJOIN

Occasionally we construct joins that are not based upon equality of values

e.g. BETWEEN CLAUSE

CUSTOMER (CUS_CODE, CUS_LNAME, CUS_ZIP, AGENT_CODE)
AGENT (AGENT_CODE, AGENT_PHONE)

Say we want a list of all CUSTOMERS whose last name are between 'W' and 'Z'. In other words, the results fall in some range

```
SELECT C.CUS_CODE, C.CUS_LNAME, C.CUS_ZIP,
   A.AGENT_CODE, A.AGENT_PHONE
FROM CUSTOMER C JOIN AGENT A
ON (C.AGENT_CODE = A. AGENT_CODE)
WHERE C.CUS_LNAME BETWEEN 'W' AND 'Z'
ORDER BY C.CUS_CODE;
```